WorkUp
Guía para terminar el proyecto

Versión académica · explicada paso a paso

> **Dónde estamos:** la parte que habla con la base de datos (carpeta `Datos`) ya está hecha para las 17 tablas.
>
> **Qué falta:** las reglas del negocio (servicios), los controllers, las pantallas (vistas) y el inicio de sesión.
>
> **Qué tiene esta guía:** 33 tareas. Cada una es una funcionalidad completa, de la pantalla a la base de datos, con la lista exacta de lo que hay que programar.

Cómo leerla

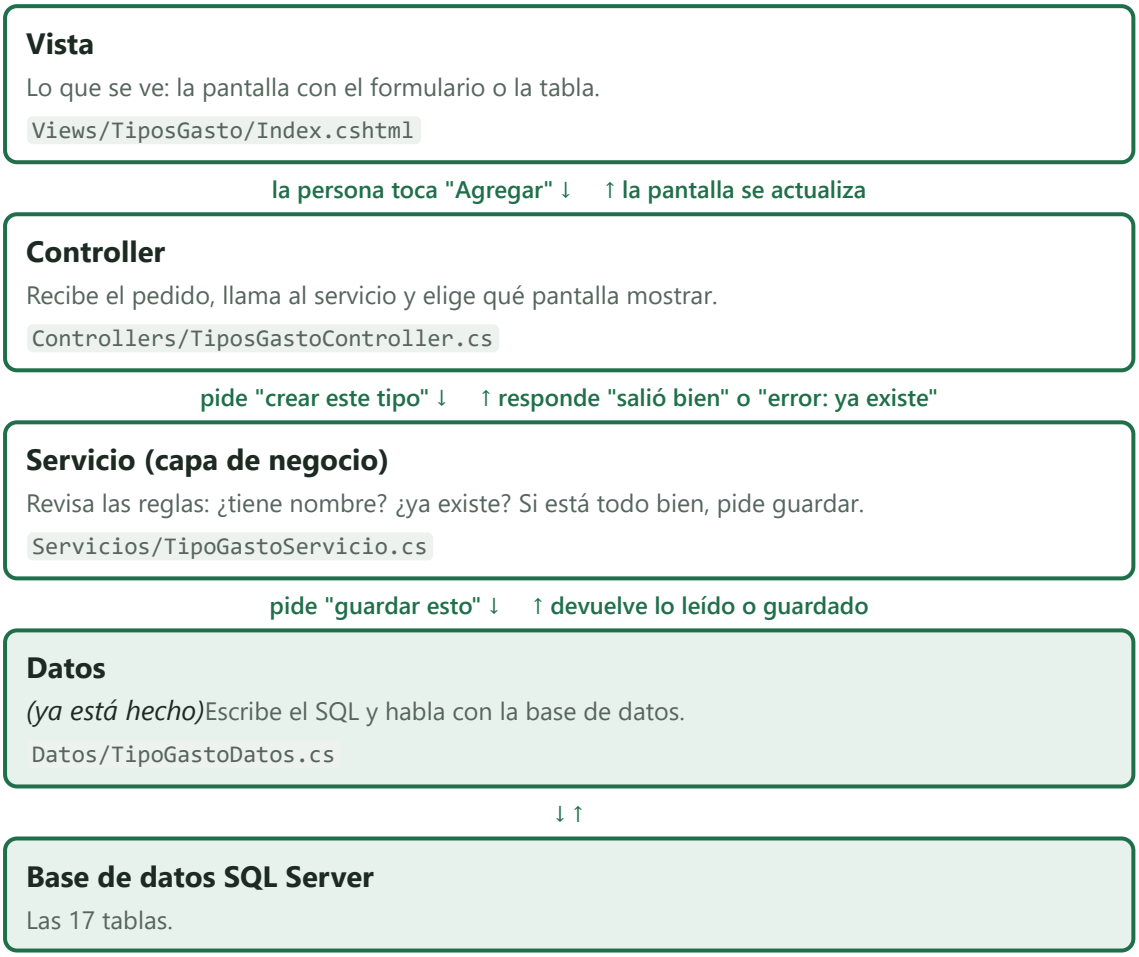
1. **Parte 1** explica cómo está armada la aplicación y las palabras técnicas. Leerla antes de programar.
2. **Parte 2** es una receta con código completo de una funcionalidad chica. Todas las demás siguen el mismo patrón.
3. **Parte 3** tiene las tareas, en el orden en que conviene hacerlas.
4. **Parte 4** es la lista para repartir el trabajo y marcar lo que está hecho.

Basada en el repositorio `ErikaRomeo/WorkUp`, rama `master` (commit `c9647fa`, 8 de octubre de 2026), y en las decisiones del equipo del 28 de septiembre de 2026. Generada el 10 de octubre de 2026.

Parte 1 · Cómo está armada la aplicación

La aplicación está dividida en **capas**, como los pisos de un edificio. Cada capa tiene un solo trabajo y solo habla con la de al lado. Así, si hay un error en una regla, se sabe que está en el servicio; si una pantalla se ve mal, se sabe que está en la vista.

Ejemplo: la dueña agrega un tipo de gasto llamado "Publicidad".



> **Regla de oro:** el número de negocio (`idNegocio`) siempre sale de la sesión de quien está conectado. Nunca de un formulario ni de la dirección web. Si viniera del formulario, alguien podría cambiar el número y ver los datos de otro local.

Palabras que se usan en la guía

Negocio	Cada local que usa WorkUp: una peluquería, un estudio de yoga. Muchos negocios usan la misma aplicación, cada uno con sus propios datos.
Capa de negocio (Servicios)	La parte del código que decide **qué está permitido**. Por ejemplo: "no se puede dar un turno en un horario ocupado". No confundir con el negocio de arriba.
idNegocio	El número que identifica a cada negocio en la base de datos. Todo lo que se guarda lleva este número para saber a qué negocio pertenece.
Sesión	El recuerdo que guarda la aplicación de quién inició sesión. Mientras la dueña está conectada, la sesión sabe su usuario y su idNegocio.

Clase de Datos	Las clases que ya existen en la carpeta `Datos`. Cada una sabe leer y guardar una tabla de la base. Solo hacen SQL, no deciden nada.
Servicio	Una clase de la carpeta `Servicios`. Recibe el pedido, revisa las reglas y le pide a las clases de Datos que guarden o lean.
Controller	Una clase de la carpeta `Controllers`. Recibe lo que la persona hace en el navegador (abrir una página, enviar un formulario), llama al servicio y elige qué pantalla mostrar.
Acción	Cada método público de un controller. Cada acción responde a una dirección (URL), por ejemplo `/Servicios/Crear`.
Vista	Un archivo `.cshtml` de la carpeta `Views`. Es la pantalla: HTML con algunos datos que pone el controller.
ViewModel	Una clase chica con exactamente los campos que tiene un formulario o una pantalla. Se usa en lugar del modelo de la tabla.
GET y POST	**GET** es abrir una página (solo mirar). **POST** es enviar un formulario (guardar o cambiar algo).
Transacción	Varias operaciones de guardado que se hacen todas juntas o ninguna. Si una falla, se deshacen todas.
UnidadDeTrabajo	La clase que vamos a crear para manejar transacciones de forma simple (tarea T01).
Hash de contraseña	La contraseña transformada en un texto que no se puede volver atrás. Se guarda eso, nunca la contraseña real.
Baja lógica	En lugar de borrar un registro, se marca como "inactivo". Así no se pierde la historia (turnos o cobros viejos que lo usan).
Anotación	La persona anotada en un turno o en una clase. Un turno individual tiene una; una clase tiene una por cada alumna.
Abono (inscripción)	El pase mensual: la clienta tiene clases fijas cada semana y paga por mes.

Reglas que valen para todas las tareas

- Cada método de un servicio recibe `idNegocio` como primer dato.
- Si una operación guarda en dos tablas o más, se usa una `UnidadDeTrabajo` (T01).
- Los servicios no tiran errores de SQL a la pantalla: devuelven un `Resultado` con un mensaje que la dueña entiende.
- Todo formulario que guarda algo es POST y lleva `[ValidateAntiForgeryToken]` en el controller.
- Cada pantalla usa su propio ViewModel, no el modelo de la tabla.
- Lo que tiene historia (servicios, trabajadoras, clases, insumos, tipos de gasto) se desactiva, no se borra.
- Probar siempre con **dos negocios** cargados: con uno solo no se nota si un negocio ve datos del otro.

Parte 2 · La receta: una funcionalidad de punta a punta

Este es el ejemplo completo de **Tipos de gasto** (tarea T09), la funcionalidad más chica. Todas las tareas de la guía se arman igual; cambian las reglas y los campos.

1. **Revisar Datos.** Fijarse qué métodos ya existen en la clase de Datos (por ejemplo `TipoGastoDatos`) y agregar los que faltan, como `ExisteNombre`.
2. **Escribir el servicio.** Un método por cada cosa que puede hacer la persona: listar, crear, modificar, cambiar estado. Primero revisa las reglas, después guarda.
3. **Escribir el controller.** Una acción GET para mostrar cada pantalla y una POST para cada formulario. Hereda de `PanelControllerBase`.
4. **Registrar el servicio** en `Program.cs` con `AddScoped`.
5. **Escribir la vista.** El archivo `.cshtml` con el formulario y la tabla.
6. **Probar** con los pasos de la tarea, con dos negocios.

Paso 2 · El servicio

```C#
// Servicios/Resultado.cs  (se hace una sola vez, en T01)
public class Resultado
{
    public bool Exito { get; private set; }
    public string? Mensaje { get; private set; }
    public string? Campo { get; private set; }

    public static Resultado Ok() => new Resultado { Exito = true };
    public static Resultado Error(string mensaje, string? campo = null)
        => new Resultado { Exito = false, Mensaje = mensaje, Campo = campo };
}

// Servicios/TipoGastoServicio.cs
public class TipoGastoServicio
{
    private readonly TipoGastoDatos datos = new TipoGastoDatos();

    public List<TipoGasto> Listar(int idNegocio)
    {
        return datos.ObtenerTipoGasto(idNegocio, false);
    }

    public Resultado Crear(int idNegocio, string nombre)
    {
        nombre = (nombre ?? "").Trim();

        // 1) Revisar las reglas
        if (nombre == "")
            return Resultado.Error("Ingrese un nombre.", "nombre");
        if (datos.ExisteNombre(idNegocio, nombre, null))
            return Resultado.Error("Ya existe un tipo de gasto con ese nombre.", "nombre");

        // 2) Guardar (el idNegocio viene de la sesión, nunca del formulario)
        datos.InsertarTipoGasto(new TipoGasto { IdNegocio = idNegocio, Nombre = nombre, Activo = true });
        return Resultado.Ok();
    }
}
```

Pasos 3 y 4 · El controller

```C#
// Controllers/TiposGastoController.cs
public class TiposGastoController : PanelControllerBase   // hereda: exige sesión y da IdNegocio
{
    private readonly TipoGastoServicio servicio;

    public TiposGastoController(TipoGastoServicio servicio)  // ASP.NET lo pasa solo
    {
        this.servicio = servicio;
    }

    [HttpGet]                       // abrir la página: /TiposGasto
    public IActionResult Index()
    {
        var tipos = servicio.Listar(IdNegocio);
        return View(tipos);         // muestra Views/TiposGasto/Index.cshtml
    }

    [HttpPost]                      // enviar el formulario
    [ValidateAntiForgeryToken]      // protección obligatoria en todo POST
    public IActionResult Crear(string nombre)
    {
        var resultado = servicio.Crear(IdNegocio, nombre);
        if (!resultado.Exito)
            TempData["Error"] = resultado.Mensaje;
        return RedirectToAction(nameof(Index));
    }
}

// Program.cs: registrar el servicio para que el controller lo reciba
builder.Services.AddScoped<TipoGastoServicio>();
```

Paso 5 · La vista

```Razor
@* Views/TiposGasto/Index.cshtml *@
@model List<WorkUp.Models.TipoGasto>

<h1>Tipos de gasto</h1>

@if (TempData["Error"] != null)
{
    <p class="text-danger" role="alert">@TempData["Error"]</p>
}

<form asp-action="Crear" method="post">   @* asp-action agrega solo la protección *@
    <label for="nombre">Nombre</label>
    <input id="nombre" name="nombre" />
    <button type="submit">Agregar</button>
</form>

<table class="table">
    <tr><th>Nombre</th><th>Estado</th></tr>
    @foreach (var tipo in Model)
    {
        <tr>
            <td>@tipo.Nombre</td>
            <td>@(tipo.Activo ? "Activo" : "Inactivo")</td>
        </tr>
    }
</table>
```

> Para formularios más grandes (Crear y Editar con varios campos) se usa un ViewModel con atributos como `[Required]`, y el controller muestra los errores con `ModelState.AddModelError(resultado.Campo, resultado.Mensaje)` en vez de `TempData`.

Parte 3 · Las tareas

Están agrupadas en módulos. Los módulos se hacen en este orden, porque cada uno usa lo anterior. Dentro de un módulo, varias tareas se pueden hacer al mismo tiempo entre distintas personas.

M0 Base técnica → M1 Negocio y cuenta de la dueña → M2 Configuración del negocio → M3 Clientes → M4 Turnos individuales → M5 Clases grupales → M6 Abonos mensuales → M7 Cobros → M8 Gastos, insumos y reportes → M9 Avisos → M10 Cuenta de cliente

Cómo leer cada tarea

Cada línea de "Qué hay que programar" tiene una etiqueta: **Datos** método nuevo o a corregir en la carpeta `Datos` · **Servicio** método de la capa de negocio · **Controller** acción · **Vista** pantalla `.cshtml` · **Otro** configuración o clase de apoyo. El cuadrado ☐ sirve para ir tachando.

Antes de empezar: arreglos pendientes en Datos

Estos errores hacen fallar algunas tareas. Conviene arreglarlos primero.

- `InscripcionDatos.InsertarInscripcion`: hoy falla al guardar porque manda valores vacíos (null). Hay que sacar del INSERT las columnas `fechaVencimiento`, `activadaEl`, `anuladaEl`, `terminadaEl` y `fechaCreacion`: la base las completa sola.
- `TrabajadorDatos.InsertarTrabajador` y `ModificarTrabajador`: mandan vacío el teléfono y el CUIL, pero la tabla los exige.
- `CuentaPropietarioDatos.InsertarCuentaPropietario`: sacar `fechaCreacion` del INSERT (la base la completa sola).
- `HorarioTrabajadorDatos.InsertarHorario`: se corrige en T08.

M0 · Base técnica

Son piezas que no se ven en pantalla pero que usan todas las demás tareas. Conviene que las haga una sola persona al principio.

T01 Guardar varias cosas juntas (UnidadDeTrabajo) y devolver errores (Resultado)

Antes hay que tener hecha: **Ninguna**

PARA QUÉ SIRVE

A veces hay que guardar en dos tablas a la vez, por ejemplo el negocio y la cuenta de la dueña. Si la segunda falla, la primera no debe quedar guardada. Además, los servicios necesitan una forma simple de decir "salió bien" o "salió mal por tal motivo".

Qué hay que programar

Otro	☐ `Servicios/Resultado.cs`	Clase con `Exito` (sí o no), `Mensaje` (el error para mostrar) y `Campo` (qué campo del formulario está mal). Ver el código de la página de la receta.
Datos	☐ `Datos/UnidadDeTrabajo.cs`	Abre una conexión, empieza una transacción y tiene un método `Confirmar()`. Si no se llama a `Confirmar()`, al terminar se deshace todo solo.
Datos	☐ Versiones "con uow" de los métodos de guardado	A cada método de Datos que se va a usar dentro de una transacción se le agrega una segunda versión que recibe `UnidadDeTrabajo uow` y usa su conexión. Lista: `InsertarNegocio`, `InsertarCuentaPropietario`, `InsertarTiposPorDefecto`, `InsertarCliente`, `InsertarTurno`, `InsertarAnotacion`, `InsertarInscripcion`, `ActivarInscripcion`, `ActualizarVencimiento`, `InsertarClaseFija`, `InsertarPago`, `VincularCuentaPorEmail`, `MarcarEmailVerificado`.

Código de ayuda

```C#
public class UnidadDeTrabajo : IDisposable
{
    internal SqlConnection Conexion { get; }
    internal SqlTransaction Transaccion { get; }

    public UnidadDeTrabajo()
    {
        Conexion = AccesoDatos.ObtenerConexion();
        Transaccion = Conexion.BeginTransaction();
    }

    public void Confirmar() => Transaccion.Commit();

    public void Dispose()
    {
        Transaccion.Dispose();   // si no se confirmó, deshace todo
        Conexion.Dispose();
    }
}

// Ejemplo de método "con uow" en NegocioDatos:
public int InsertarNegocio(Negocio negocio, UnidadDeTrabajo uow)
{
    using (SqlCommand cmd = new SqlCommand(consulta, uow.Conexion, uow.Transaccion))
    {
        // ... mismos parámetros que el método original ...
        return (int)cmd.ExecuteScalar()!;
    }
}
```

Cómo probar que funciona

✓ Registrar un negocio con un usuario que ya existe (cuando esté T03).

✓ Revisar la tabla `Negocios`: el negocio nuevo no tiene que estar.

T02 Inicio de sesión con cookie y controller base del panel

Antes hay que tener hecha: **Ninguna**

PARA QUÉ SIRVE

La aplicación tiene que recordar quién está conectado. Cuando la dueña inicia sesión, se guarda una "cookie" (un dato en el navegador) con su usuario y el número de su negocio. Todas las pantallas del panel leen el número de negocio de ahí.

Qué hay que programar

Otro	☐ `Program.cs`	Activar la autenticación con cookie y agregar `app.UseAuthentication()` justo antes de `app.UseAuthorization()`. Ahí también se registra cada servicio nuevo con `builder.Services.AddScoped<NombreServicio>();`.
Controller	☐ `Controllers/PanelControllerBase.cs`	Un controller "padre" del que heredan todos los del panel. Exige sesión de dueña y ofrece la propiedad `IdNegocio`, leída de la cookie.

Vista	☐ `Views/Shared/_Layout.cshtml`	Agregar el menú del panel (Agenda, Clases, Clientes, Abonos, Cobros, Gastos, Insumos, Reportes, Configuración). Se muestra solo si hay una dueña conectada.

> **Importante:** El idNegocio **nunca** se pone en un formulario, ni siquiera oculto. Si viniera del formulario, cualquiera podría cambiar el número y ver datos de otro negocio.

Código de ayuda

```C#
// Program.cs
builder.Services
    .AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(opciones =>
    {
        opciones.LoginPath = "/Cuenta/IniciarSesion";
        opciones.AccessDeniedPath = "/Cuenta/IniciarSesion";
    });
// ...
app.UseAuthentication();
app.UseAuthorization();

// Controllers/PanelControllerBase.cs
[Authorize(Roles = "Propietario")]
public abstract class PanelControllerBase : Controller
{
    protected int IdNegocio => int.Parse(User.FindFirst("IdNegocio")!.Value);
}
```

Cómo probar que funciona

✓ Sin iniciar sesión, escribir en el navegador la dirección de cualquier pantalla del panel.

✓ Tiene que llevar a la pantalla de Iniciar sesión.

M1 · Negocio y cuenta de la dueña

Es la puerta de entrada: sin esto nadie puede usar el panel.

T03 Registrar un negocio nuevo

Antes hay que tener hecha: **T01, T02**

PARA QUÉ SIRVE

Una persona entra a la página de registro, escribe los datos de su local, elige un usuario y una contraseña, y queda adentro del panel. En la base se guardan, juntos, el negocio, la cuenta y los 6 tipos de gasto que vienen cargados de entrada.

Qué hay que programar

Datos	☐ `CuentaPropietarioDatos.ExisteUsuario(usuario)`	Nuevo. Devuelve si ese usuario ya está tomado.
Servicio	☐ `CuentaPropietarioServicio.RegistrarNegocio(modelo)`	Revisa los datos (ver reglas). Convierte la contraseña en hash. Abre una `UnidadDeTrabajo`, guarda el negocio, la cuenta y los tipos de gasto por defecto, y confirma.
Controller	☐ `CuentaController.Registrar()` (GET)	Muestra el formulario vacío.
Controller	☐ `CuentaController.Registrar(modelo)` (POST)	Llama al servicio. Si hay error, vuelve a mostrar el formulario con el mensaje. Si sale bien, inicia la sesión y va al panel.
Vista	☐ `Views/Cuenta/Registrar.cshtml`	Formulario: nombre del local, dirección web corta (slug), teléfono, email para avisos (opcional), días de gracia, usuario, contraseña y repetir contraseña.
Otro	☐ `RegistroNegocioViewModel`	Clase con los campos del formulario.

Reglas que tiene que revisar el servicio

- Nombre, teléfono y usuario son obligatorios.
- El slug (la parte de la dirección web, como `peluqueria-marina`) solo puede tener minúsculas, números y guiones, y no puede estar usado por otro negocio.
- La contraseña tiene 8 caracteres o más y las dos que se escriben son iguales.
- La contraseña se guarda con `PasswordHasher`, nunca como texto.

Cómo probar que funciona

✓ Registrar un negocio: entra al panel y en la tabla `TiposDeGasto` aparecen 6 tipos para ese negocio.

✓ Registrar otro con el mismo usuario: aparece el error y no se guarda nada.

T04 Iniciar y cerrar sesión

Antes hay que tener hecha: **T02, T03**

PARA QUÉ SIRVE

La dueña escribe su usuario y contraseña para entrar, y tiene un botón para salir. También puede cambiar su contraseña.

Qué hay que programar

Servicio	☐ `CuentaPropietarioServicio.ValidarCredenciales(usuario, contrasena)`	Busca la cuenta y compara la contraseña con el hash. Si algo no coincide, devuelve nada (null).
Servicio	☐ `CuentaPropietarioServicio.CambiarContrasena(idCuenta, actual, nueva)`	Verifica la actual y guarda la nueva.
Controller	☐ `CuentaController.IniciarSesion()` GET y POST	Muestra el formulario; al enviarlo, valida y crea la cookie de sesión.
Controller	☐ `CuentaController.CerrarSesion()` POST	Borra la cookie y vuelve al inicio.
Controller	☐ `CuentaController.CambiarContrasena()` GET y POST	Formulario con contraseña actual, nueva y repetir.
Vista	☐ `Views/Cuenta/IniciarSesion.cshtml`, `Views/Cuenta/CambiarContrasena.cshtml`	

Reglas que tiene que revisar el servicio

- Si el usuario no existe o la contraseña está mal, el mensaje es el mismo: "Usuario o contraseña incorrectos". Así nadie puede averiguar qué usuarios existen.

Código de ayuda

```C#
// Dentro de CuentaController.IniciarSesion (POST), si las credenciales son correctas:
var claims = new List<Claim>
{
    new Claim(ClaimTypes.NameIdentifier, cuenta.IdCuentaPropietario.ToString()),
    new Claim(ClaimTypes.Name, cuenta.Usuario),
    new Claim(ClaimTypes.Role, "Propietario"),
    new Claim("IdNegocio", cuenta.IdNegocio.ToString())
};
var identidad = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
await HttpContext.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme,
                              new ClaimsPrincipal(identidad));
return RedirectToAction("Index", "Panel");
```

Cómo probar que funciona

✓ Crear dos negocios. Entrar con cada uno: cada uno ve solo sus datos.

✓ Cerrar sesión y tocar "atrás" en el navegador: no debe dejar seguir usando el panel.

T05 Editar los datos del negocio

Antes hay que tener hecha: **T04**

PARA QUÉ SIRVE

La dueña puede corregir el nombre del local, el slug, el teléfono, el email de avisos y los días de gracia del abono.

Qué hay que programar

Servicio	☐ `NegocioServicio.ObtenerNegocio(idNegocio)`	Trae los datos del negocio conectado.
Servicio	☐ `NegocioServicio.Modificar(idNegocio, modelo)`	Mismas reglas que el registro. El slug no puede ser el de otro negocio (`ExisteSlug(slug, idNegocio)`).
Controller	☐ `NegocioController.Editar()` GET y POST	Hereda de `PanelControllerBase`.
Vista	☐ `Views/Negocio/Editar.cshtml`	

Cómo probar que funciona

✓ Cambiar el slug y comprobar que la dirección pública de reservas (T16) cambia.

M2 · Configuración del negocio

Lo que la dueña carga una vez y después usa todos los días: qué servicios ofrece, cuánto cobra, quién trabaja y en qué horario. Las cuatro tareas siguen el mismo patrón de la receta y se pueden repartir entre varias personas.

T06 Lista de servicios y precios (tarifario)

Antes hay que tener hecha: **T04**

PARA QUÉ SIRVE

La dueña carga lo que ofrece. Por ejemplo, "Corte de pelo, $8.000, 45 minutos" (individual) o "Yoga, $5.000, 60 minutos, 12 personas" (clase).

Qué hay que programar

Datos	☐ `ServicioDatos.ExisteNombre(idNegocio, nombre, excluirId)`	Nuevo. Evita dos servicios con el mismo nombre.
Servicio	☐ `TarifarioServicio.Listar(idNegocio, soloActivos)`	La lista para la tabla.
Servicio	☐ `TarifarioServicio.Obtener(idNegocio, idServicio)`	Uno solo, para editar.
Servicio	☐ `TarifarioServicio.Crear(idNegocio, modelo)`	Revisa las reglas y guarda.
Servicio	☐ `TarifarioServicio.Modificar(idNegocio, idServicio, modelo)`	Igual que crear, pero actualiza.
Servicio	☐ `TarifarioServicio.CambiarEstado(idNegocio, idServicio, activo)`	Activa o desactiva (baja lógica).
Controller	☐ `ServiciosController`: `Index`, `Crear` (GET/POST), `Editar` (GET/POST), `CambiarEstado` (POST)	
Vista	☐ `Views/Servicios/Index.cshtml`, `Crear.cshtml`, `Editar.cshtml`	Crear y Editar pueden compartir un mismo formulario parcial `_Formulario.cshtml`.

Reglas que tiene que revisar el servicio

- Nombre obligatorio y que no se repita en el negocio.
- Precio mayor a 0. Duración entre 5 y 480 minutos.
- Capacidad 1 si es individual, más de 1 si es una clase.

Cómo probar que funciona

✓ Desactivar un servicio: ya no aparece para dar turnos ni para armar la grilla.

T07 Precios del abono mensual

Antes hay que tener hecha: **T04**

PARA QUÉ SIRVE
La dueña define cuánto cuesta el abono según cuántas clases por semana incluye: 1 por semana $20.000, 2 por semana $35.000, y así.

Qué hay que programar

Servicio	☐ `PrecioPaseServicio.Listar(idNegocio)`	
Servicio	☐ `PrecioPaseServicio.Crear(idNegocio, frecuencia, precio)`	La frecuencia va de 1 a 7 y no puede repetirse.
Servicio	☐ `PrecioPaseServicio.Modificar(idNegocio, idPrecioPase, precio)`	Solo cambia el precio.
Servicio	☐ `PrecioPaseServicio.Eliminar(idNegocio, idPrecioPase)`	
Controller	☐ `PreciosPaseController`: `Index`, `Crear` (POST), `Editar` (GET/POST), `Eliminar` (POST)	
Vista	☐ `Views/PreciosPase/Index.cshtml`, `Editar.cshtml`	En Index va la tabla y un formulario corto para agregar.

Cómo probar que funciona

✓ Sin precio para "3 por semana" no se puede inscribir a nadie a ese abono (T21).

T08 Trabajadoras y sus horarios

Antes hay que tener hecha: **T04**

PARA QUÉ SIRVE
La dueña carga a las personas que trabajan y en qué días y horas atienden, incluidas las pausas (por ejemplo, de 13 a 14 almuerza). Esos horarios deciden qué turnos se pueden dar.

Qué hay que programar

Datos	☐ `TrabajadorDatos.ExisteCuil(idNegocio, cuil, excluirId)`	Nuevo.
Datos	☐ Corregir `HorarioTrabajadorDatos.InsertarHorario`	Que reciba `idNegocio` y no guarde si la trabajadora es de otro negocio.
Servicio	☐ `TrabajadorServicio.Listar`, `ObtenerConHorarios`, `Crear`, `Modificar`, `CambiarEstado`	Siempre con `idNegocio` como primer dato.
Servicio	☐ `TrabajadorServicio.AgregarHorario(idNegocio, horario)`	Revisa que el horario tenga sentido (ver reglas).

Servicio	☐ `TrabajadorServicio.EliminarHorario(idNegocio, idHorario)`	
Controller	☐ `TrabajadoresController`: `Index`, `Crear`, `Editar`, `CambiarEstado`	
Controller	☐ `TrabajadoresController.Horarios(id)` GET, `AgregarHorario` POST, `EliminarHorario` POST	La pantalla de la semana de una trabajadora.
Vista	☐ `Views/Trabajadores/Index.cshtml`, `Crear.cshtml`, `Editar.cshtml`, `Horarios.cshtml`	En Horarios, una tabla de lunes a domingo con las pausas marcadas.

Reglas que tiene que revisar el servicio

- Nombre, apellido, teléfono y CUIL son obligatorios. El CUIL tiene 11 números y no se repite en el negocio.
- La hora de fin es mayor que la de inicio.
- Dos tramos de trabajo del mismo día no se pisan.
- Una pausa tiene que estar dentro de un tramo de trabajo.

Cómo probar que funciona

✓ Cargar lunes de 9 a 18 con pausa de 13 a 14. En T11, a las 13 no se ofrecen turnos.

T09 Tipos de gasto

Antes hay que tener hecha: **T03**

PARA QUÉ SIRVE
Las categorías de los gastos: alquiler, luz, insumos, etc. El negocio arranca con 6 cargados y la dueña puede agregar, renombrar o desactivar.

Qué hay que programar

Datos	☐ `TipoGastoDatos.ExisteNombre(idNegocio, nombre, excluirId)`	Nuevo.
Servicio	☐ `TipoGastoServicio.Listar`, `Crear`, `Modificar`, `CambiarEstado`	Es la receta completa de la Parte 2.
Controller	☐ `TiposGastoController`: `Index`, `Crear`, `Editar`, `CambiarEstado`	
Vista	☐ `Views/TiposGasto/Index.cshtml`, `Editar.cshtml`	

Cómo probar que funciona

✓ Un tipo desactivado no aparece al cargar un gasto nuevo, pero los gastos viejos lo siguen mostrando.

M3 · Clientes

Cada turno, clase, abono y cobro va a nombre de una ficha de cliente.

T10 Fichas de clientes

Antes hay que tener hecha: **T04**

PARA QUÉ SIRVE

La dueña busca clientes, crea fichas nuevas, las edita y ve el historial de cada una: turnos, clases, abonos y pagos. La ficha tiene observaciones de salud (alergias, lesiones) que se muestran al atender.

Qué hay que programar

Servicio	☐ `ClienteServicio.Buscar(idNegocio, texto)`	Sin texto trae todos; con texto busca por nombre, apellido, email o teléfono (`BuscarClientes`).
Servicio	☐ `ClienteServicio.Crear(idNegocio, modelo)`	Revisa reglas y guarda.
Servicio	☐ `ClienteServicio.Modificar(idNegocio, idCliente, modelo)`	
Servicio	☐ `ClienteServicio.ObtenerFicha(idNegocio, idCliente)`	Junta datos del cliente + `ObtenerAnotacionesPorCliente` + `ObtenerInscripcionesPorCliente` + `ObtenerPagosPorCliente`.
Controller	☐ `ClientesController`: `Index(texto)`, `Crear`, `Editar`, `Detalle(id)`	
Vista	☐ `Views/Clientes/Index.cshtml`, `Crear.cshtml`, `Editar.cshtml`, `Detalle.cshtml`	En Detalle, botones "Nuevo turno" e "Inscribir a abono".

Reglas que tiene que revisar el servicio

- Nombre, apellido y teléfono obligatorios.
- El email es opcional, pero si se escribe tiene que ser válido y no repetirse en el negocio.
- La dueña nunca vincula la ficha a una cuenta de cliente: eso pasa solo, en T31.

Cómo probar que funciona

✓ Buscar por una parte del teléfono y encontrar a la clienta.

M4 · Turnos individuales

Es lo más importante de la aplicación: la agenda del día. T11 es la tarea más difícil del proyecto; conviene que la haga alguien con más práctica o en pareja.

T11 Calcular los horarios libres

Antes hay que tener hecha: **T06, T08**

PARA QUÉ SIRVE

Para dar un turno hay que saber qué horarios están libres. Ejemplo: Marina trabaja el lunes de 9 a 13, el corte dura 45 minutos y ya tiene un turno a las 10:30. Los horarios libres son 9:00, 9:45 (termina 10:30) y 11:15 (termina 12:00). Las 12:00 también, porque termina justo a las 12:45.

Qué hay que programar

Servicio	☐ `DisponibilidadServicio.ObtenerHorariosLibres(idNegocio, idServicio, idTrabajador, fecha)`	Devuelve la lista de horarios de inicio posibles. Ver los pasos abajo.
Servicio	☐ `DisponibilidadServicio.EstaLibre(idNegocio, idServicio, idTrabajador, comienzaEl)`	Responde sí o no para un horario puntual. Se usa justo antes de guardar, por si otra persona lo tomó en el medio.

Paso a paso

1. Traer los tramos de trabajo de ese día (`HorarioTrabajadorDatos.ObtenerHorariosPorDia`) y quitarles las pausas.
2. Dentro de cada tramo, armar horarios de inicio cada tantos minutos como dure el servicio. El turno tiene que terminar antes de que termine el tramo.
3. Traer los turnos que la trabajadora ya tiene ese día (`TurnoDatos.ObtenerTurnosPorTrabajadorYDia`) y sacar los horarios que se pisan con alguno. Los turnos suspendidos no cuentan.
4. Si la fecha es hoy, sacar los horarios que ya pasaron.

Cómo probar que funciona

✓ Día en que la trabajadora no trabaja: lista vacía.
✓ Día con pausa: no se ofrecen horarios dentro de la pausa.
✓ Con un turno ya dado: ese horario no aparece.

T12 Agenda del día y detalle de un turno

Antes hay que tener hecha: **T10**

PARA QUÉ SIRVE

La dueña ve todos los turnos y clases de un día, ordenados por hora y agrupados por trabajadora, y puede entrar a cada uno para ver los datos del cliente.

Qué hay que programar

Servicio	☐ `AgendaServicio.ObtenerDia(idNegocio, fecha)`	Usa `TurnoDatos.ObtenerTurnosPorFecha` y agrega servicio, trabajadora y cliente de cada turno.
Servicio	☐ `AgendaServicio.ObtenerDetalle(idNegocio, idTurno)`	Usa `ObtenerTurnoConAnotaciones`. Muestra observaciones de salud y si ya se cobró (`ObtenerPagoDeAnotacion`).
Controller	☐ `AgendaController.Index(fecha)`	Sin fecha muestra hoy. Botones día anterior y siguiente.
Controller	☐ `AgendaController.Detalle(id)`	
Vista	☐ `Views/Agenda/Index.cshtml`, `Views/Agenda/Detalle.cshtml`	

Cómo probar que funciona

✓ Con dos negocios cargados, la agenda de uno nunca muestra turnos del otro.

T13 Dar un turno desde el panel

Antes hay que tener hecha: **T01, T11, T12**

PARA QUÉ SIRVE
Una clienta llama por teléfono y la dueña le da un turno. Si es nueva, la dueña carga sus datos ahí mismo.

Qué hay que programar

Servicio	☐ `TurnoServicio.AgendarIndividual(idNegocio, modelo)`	Revisa todo, y en una `UnidadDeTrabajo` guarda el cliente (si es nuevo), el turno y la anotación.
Controller	☐ `AgendaController.NuevoTurno()` GET	Formulario: servicio, trabajadora, fecha, horario y cliente (buscar uno o cargar uno nuevo).
Controller	☐ `AgendaController.HorariosLibres(idServicio, idTrabajador, fecha)` GET	Devuelve solo los botones de horario. La página lo pide cuando cambia el servicio, la trabajadora o la fecha.
Controller	☐ `AgendaController.NuevoTurno(modelo)` POST	Si sale bien, va al detalle del turno.
Vista	☐ `Views/Agenda/NuevoTurno.cshtml` y `Views/Agenda/_HorariosLibres.cshtml`	El segundo es una vista parcial (un pedazo de pantalla).

Reglas que tiene que revisar el servicio

- El servicio es individual, está activo y es de este negocio. Lo mismo la trabajadora.
- `EstaLibre` tiene que dar sí justo antes de guardar.
- El turno se guarda con `Tipo = Individual`, `Origen = Reservado`, `Estado = Activo` y la hora de fin calculada (inicio + duración). La anotación queda `Reservada`.

Cómo probar que funciona

✓ Abrir el formulario en dos pestañas, elegir el mismo horario y guardar las dos: la segunda tiene que decir que ese horario ya no está libre.

T14 Atender a alguien que llegó sin turno

Antes hay que tener hecha: **T13**

PARA QUÉ SIRVE
Entra alguien al local sin reserva y lo atienden en el momento. Igual se registra, para que figure en la agenda y se pueda cobrar.

Qué hay que programar

Servicio	☐ `TurnoServicio.RegistrarSinTurno(idNegocio, modelo)`	Como T13, pero empieza ahora, con `Origen = SinTurno` y la anotación directamente `Atendida`.
Controller	☐ `AgendaController.SinTurno()` GET y POST	Después de guardar ofrece cobrar (T24).
Vista	☐ `Views/Agenda/SinTurno.cshtml`	

Cómo probar que funciona

✓ Aparece en la agenda de hoy marcado como "sin turno".

T15 Confirmar, cancelar y marcar si vino

Antes hay que tener hecha: **T12**

PARA QUÉ SIRVE
Cada anotación pasa por estados: Reservada, Confirmada, Atendida (vino), Ausente (no vino) o Cancelada. No todos los cambios están permitidos.

Qué hay que programar

Servicio	☐ `AnotacionServicio.CambiarEstado(idNegocio, idAnotacion, nuevoEstado)`	Revisa que el cambio esté permitido (ver reglas) y llama a `AnotacionDatos.CambiarEstadoAnotacion`.
Controller	☐ `AgendaController.Confirmar(idAnotacion)`, `Cancelar(idAnotacion)`, `MarcarAsistencia(idAnotacion, vino)`	Los tres son POST y vuelven al detalle del turno.
Vista	☐ Botones en `Views/Agenda/Detalle.cshtml`	Solo se muestran los botones que corresponden al estado actual.

Reglas que tiene que revisar el servicio

- Desde Reservada se puede pasar a Confirmada, Cancelada, Atendida o Ausente.
- Desde Confirmada se puede pasar a Cancelada, Atendida o Ausente.
- Atendida, Ausente y Cancelada ya no cambian más.
- No se puede marcar Atendida o Ausente antes de la hora del turno.
- No hay "reprogramar": se cancela y se da otro turno (lo decidió el equipo).

Cómo probar que funciona

✓ Cancelar un turno y ver que ese horario vuelve a aparecer libre.

T16 Reserva desde la página pública

Antes hay que tener hecha: **T11, T13**

PARA QUÉ SIRVE

Cualquier persona entra a la dirección del negocio, por ejemplo `/reservar/peluqueria-marina`, elige servicio, trabajadora y horario, deja sus datos y reserva. No necesita cuenta.

Qué hay que programar

Servicio	☐ `ReservaPublicaServicio.ObtenerNegocio(slug)`	Si el slug no existe, la página muestra "no encontrado".
Servicio	☐ `ReservaPublicaServicio.ObtenerOpciones(idNegocio)`	Servicios individuales activos y trabajadoras activas.
Servicio	☐ `ReservaPublicaServicio.Reservar(slug, modelo, idCuentaCliente)`	Busca la ficha por email en ese negocio (`ObtenerClientePorEmail`); si no existe, la crea. Después hace lo mismo que `AgendarIndividual`.
Controller	☐ `ReservarController.Index(slug)`	Lleva `[AllowAnonymous]` porque no hace falta sesión.
Controller	☐ `ReservarController.Horarios(slug, idServicio, idTrabajador, fecha)`	Los botones de horario (reusa T11).
Controller	☐ `ReservarController.Confirmar(slug, ...)` GET y POST	Pide nombre, apellido, email y teléfono.
Controller	☐ `ReservarController.Listo(slug, id)`	Resumen del turno reservado.
Vista	☐ `Views/Reservar/Index.cshtml`, `_Horarios.cshtml`, `Confirmar.cshtml`, `Listo.cshtml`	
Otro	☐ `Program.cs`	Agregar la ruta: `app.MapControllerRoute("reservar", "reservar/{slug}/{action=Index}/{id?}", new { controller = "Reservar" });`

Cómo probar que funciona

✓ Reservar desde una ventana de incógnito y ver el turno en la agenda de la dueña.

M5 · Clases grupales

Las clases se repiten todas las semanas (la grilla), tienen un cupo máximo y pueden tener alumnas fijas (con abono) y sueltas (pagan esa clase).

T17 Grilla semanal de clases

Antes hay que tener hecha: **T06, T08**

PARA QUÉ SIRVE
La dueña arma el cartel de horarios: "Yoga, lunes y miércoles 18:00, con Laura". Eso se repite todas las semanas.

Qué hay que programar

Servicio	☐ `GrillaServicio.Listar(idNegocio, soloActivas)`	Agrupada por día y hora.
Servicio	☐ `GrillaServicio.Crear(idNegocio, modelo)` y `Modificar(...)`	Revisan las reglas.
Servicio	☐ `GrillaServicio.CambiarEstado(idNegocio, idClaseGrilla, activa)`	No se puede desactivar si hay abonos activos que la usan.
Controller	☐ `GrillaController`: `Index`, `Crear`, `Editar`, `CambiarEstado`	
Vista	☐ `Views/Grilla/Index.cshtml`, `Crear.cshtml`, `Editar.cshtml`	Index con una columna por día, de lunes a domingo.

Reglas que tiene que revisar el servicio

- El servicio es una clase (capacidad mayor a 1) y está activo.
- La clase cae dentro del horario de trabajo de la trabajadora ese día.
- La trabajadora no tiene otra clase a la misma hora.

Cómo probar que funciona

✓ La grilla se ve igual que un cartel de horarios en la pared del local.

T18 Clases de la semana y lista de cada clase

Antes hay que tener hecha: **T15, T17**

PARA QUÉ SIRVE
La dueña ve las clases de una semana con cuántos lugares quedan, y entra a cada una para ver quiénes vienen y tomar asistencia.

> Importante: la grilla dice "Yoga los lunes", pero cada lunes concreto es un turno distinto en la tabla `Turnos`. Ese turno se crea la primera vez que alguien abre o usa esa clase. En ese momento se anotan solas las alumnas fijas (las que tienen abono en esa clase).

Qué hay que programar

Datos	☐ `ClaseFijaInscripcionDatos.ObtenerFijosVigentes(idClaseGrilla, idNegocio, fecha)`	Nuevo. Las alumnas con abono pendiente o activo en esa clase.
Datos	☐ `AnotacionDatos.ExisteAnotacionActiva(idTurno, idCliente, idNegocio)`	Nuevo. Para no anotar dos veces a la misma persona.
Servicio	☐ `ClaseServicio.ObtenerSemana(idNegocio, lunes)`	Para cada clase de la grilla y cada día, muestra los lugares libres. No guarda nada.
Servicio	☐ `ClaseServicio.ObtenerOCrearClase(idNegocio, idClaseGrilla, fecha)`	Busca el turno de esa fecha (`ObtenerClasePorGrillaYFecha`). Si no existe, en una `UnidadDeTrabajo` crea el turno y una anotación por cada alumna fija.
Servicio	☐ `ClaseServicio.ObtenerLista(idNegocio, idClaseGrilla, fecha)`	Llama al anterior y devuelve las alumnas con su estado.
Controller	☐ `ClasesController.Semana(lunes)`	La semana actual si no se indica.
Controller	☐ `ClasesController.Lista(idClaseGrilla, fecha)`	Con los botones de asistencia de T15.
Vista	☐ `Views/Clases/Semana.cshtml`, `Views/Clases/Lista.cshtml`	

Cómo probar que funciona

✓ Abrir dos veces la misma clase: en la tabla `Turnos` hay un solo registro para esa fecha.

✓ Los lugares libres nunca dan un número negativo.

T19 Anotar a alguien en una clase suelta

Antes hay que tener hecha: **T18**

PARA QUÉ SIRVE

Una clienta sin abono quiere venir a una clase puntual.

Qué hay que programar

Servicio	☐ `ClaseServicio.AnotarSuelta(idNegocio, idClaseGrilla, fecha, idCliente)`	Revisa las reglas y guarda la anotación sin abono (`IdInscripcion` vacío).
Controller	☐ `ClasesController.AnotarSuelta(...)` POST	Desde la lista de la clase, con un buscador de clientes. Después ofrece cobrar (T24).
Vista	☐ Formulario dentro de `Views/Clases/Lista.cshtml`	

Reglas que tiene que revisar el servicio

- La clase no pasó y no está suspendida.

- Hay lugar: capacidad menos anotaciones activas (`ContarAnotacionesActivas`) es mayor a 0.
- La clienta no está ya anotada.

Cómo probar que funciona

✓ Con la clase llena, aparece "No hay lugar en esta clase".

T20 Suspender una clase

Antes hay que tener hecha: **T18**

PARA QUÉ SIRVE
Un feriado o una ausencia: la dueña suspende la clase de ese día y la aplicación le muestra a quién avisar.

Qué hay que programar

Servicio	☐ `ClaseServicio.Suspender(idNegocio, idTurno, motivo)`	Usa `TurnoDatos.SuspenderTurno` y devuelve la lista de alumnas anotadas para avisarles (T29).
Controller	☐ `ClasesController.Suspender(idTurno, motivo)` POST	
Vista	☐ Botón y campo de motivo en `Views/Clases/Lista.cshtml`	

Reglas que tiene que revisar el servicio

- El motivo es obligatorio.
- Una clase suspendida no cuenta como ausencia de nadie.

Cómo probar que funciona

✓ La clase aparece tachada en la semana y en la agenda.

M6 · Abonos mensuales

El abono tiene dos pasos separados (decisión del equipo): primero se inscribe a la clienta, que queda "pendiente", y después se cobra, cosa que lo activa.

T21 Inscribir a una clienta a un abono

Antes hay que tener hecha: **T07, T17**

PARA QUÉ SIRVE

Ana quiere venir a yoga lunes y miércoles. La dueña la inscribe en el abono de 2 por semana y elige esas dos clases. Ana ya ocupa su lugar fijo, pero el abono queda pendiente hasta que pague.

Qué hay que programar

Servicio	☐ `InscripcionServicio.Inscribir(idNegocio, modelo)`	Revisa las reglas, y en una `UnidadDeTrabajo` guarda la inscripción (Pendiente) y una clase fija por cada clase elegida.
Servicio	☐ `InscripcionServicio.ObtenerDetalle(idNegocio, idInscripcion)`	Con clases fijas, pagos y vencimiento.
Servicio	☐ `InscripcionServicio.Listar(idNegocio, estado)`	
Controller	☐ `AbonosController.Index(estado)`	Pestañas: pendientes, activos y vencidos.
Controller	☐ `AbonosController.Inscribir(idCliente)` GET y POST	Elegir cuántas por semana (muestra el precio) y las clases con lugar.
Controller	☐ `AbonosController.Detalle(id)`	
Vista	☐ `Views/Abonos/Index.cshtml`, `Inscribir.cshtml`, `Detalle.cshtml`	

Reglas que tiene que revisar el servicio

- La clienta no tiene otro abono pendiente o activo.
- Existe un precio para esa cantidad de clases por semana.
- Elige exactamente tantas clases como su abono, en días distintos.
- Cada clase elegida tiene lugar para una fija más (`ContarFijosVigentes`).

Cómo probar que funciona

✓ Si falla al guardar la segunda clase fija, no queda la inscripción a medias.

T22 Cobrar el abono

Antes hay que tener hecha: **T21**

PARA QUÉ SIRVE

Ana paga el mes. Si el abono estaba pendiente se activa y vence en un mes. Si ya estaba activo, se le suma un mes más.

Qué hay que programar

Servicio	☐ `InscripcionServicio.CobrarPase(idNegocio, idInscripcion, modelo)`	En una `UnidadDeTrabajo` guarda el pago (concepto `Pase`) y activa el abono o le corre el vencimiento.
Controller	☐ `AbonosController.Cobrar(id)` GET y POST	
Vista	☐ `Views/Abonos/Cobrar.cshtml`	Muestra el importe sugerido, que se puede cambiar.

Reglas que tiene que revisar el servicio

- El importe sugerido es el precio del abono. Si se cobra otro importe, hay que escribir el motivo.
- Pendiente: se activa (`ActivarInscripcion`) y vence en la fecha de pago más un mes.
- Activo: el nuevo vencimiento es un mes después del vencimiento actual (o de hoy, si ya venció) (`ActualizarVencimiento`).

Cómo probar que funciona

✓ El pago aparece en la caja del día (T25) y en el reporte (T27).

T23 Abonos vencidos, baja y anulación

Antes hay que tener hecha: **T22**

PARA QUÉ SIRVE
La dueña ve quién no pagó a tiempo (pasados los días de gracia) y decide si le da de baja. El lugar nunca se libera solo.

Qué hay que programar

Datos	☐ `InscripcionDatos.ObtenerVencidas(idNegocio, hoy)`	Nuevo: abonos activos cuyo vencimiento más los días de gracia ya pasó.
Servicio	☐ `InscripcionServicio.ObtenerVencidas(idNegocio)`	Con días de atraso y teléfono.
Servicio	☐ `InscripcionServicio.Terminar(idNegocio, idInscripcion)`	Solo abonos activos. Libera sus lugares fijos (`TerminarInscripcion`).
Servicio	☐ `InscripcionServicio.Anular(idNegocio, idInscripcion)`	Solo abonos pendientes que nunca se pagaron (`AnularInscripcion`).
Controller	☐ `AbonosController.Vencidos()`, `Terminar(id)` POST, `Anular(id)` POST	Los POST piden confirmación.
Vista	☐ `Views/Abonos/Vencidos.cshtml`	

Cómo probar que funciona

✓ Terminar un abono y ver que se libera un lugar en sus clases.

M7 · Cobros

Todo lo que entra de plata queda registrado. Un cobro mal cargado no se borra: se anula con un motivo.

T24 Cobrar un turno, una clase suelta o una atención sin turno

Antes hay que tener hecha: **T13**

PARA QUÉ SIRVE
Al terminar la atención la dueña registra lo que cobró.

Qué hay que programar

Servicio	☐ `CobroServicio.CobrarAnotacion(idNegocio, idAnotacion, modelo)`	Revisa las reglas, elige el concepto y guarda el pago (`PagoDatos.InsertarPago`).
Controller	☐ `CobrosController.Cobrar(idAnotacion)` GET y POST	Se llega desde el detalle del turno o la lista de la clase.
Vista	☐ `Views/Cobros/Cobrar.cshtml`	

Reglas que tiene que revisar el servicio

- La anotación es de este negocio, no está cancelada y no tiene ya un pago (`ObtenerPagoDeAnotacion`).
- Concepto: `Turno` si fue un turno reservado, `SinTurno` si llegó sin turno, `ClaseSuelta` si fue una clase sin abono.
- Importe sugerido: el precio del servicio. Si cambia, hay que escribir el motivo.
- La fecha de pago se puede cambiar (define en qué mes cuenta). La fecha de registro la pone el sistema.

Cómo probar que funciona

✓ Intentar cobrar dos veces lo mismo: la segunda vez no deja.

T25 Caja del día y anular un cobro

Antes hay que tener hecha: **T24**

PARA QUÉ SIRVE
La dueña ve todo lo que cobró en un día y el total. Si cargó algo mal, lo anula escribiendo por qué.

Qué hay que programar

Servicio	☐ `CobroServicio.ObtenerCaja(idNegocio, dia)`	Cobros del día (`ObtenerPagosPorDia`) y el total sin contar los anulados.
Servicio	☐ `CobroServicio.Anular(idNegocio, idPago, motivo)`	Motivo obligatorio. No se puede anular dos veces (`AnularPago`).
Controller	☐ `CobrosController.Index(dia)`, `Anular(id)` GET y POST	Index muestra hoy si no se indica el día.
Vista	☐ `Views/Cobros/Index.cshtml`, `Views/Cobros/Anular.cshtml`	

Cómo probar que funciona

✓ Un cobro anulado se sigue viendo, tachado y con su motivo, y no suma al total.

M8 · Gastos, insumos y reportes

Cierra las cuentas del negocio: cuánto entró, cuánto salió y qué falta comprar.

T26 Gastos e insumos

Antes hay que tener hecha: **T09**

PARA QUÉ SIRVE

La dueña anota cada gasto con su tipo. También lleva la lista de insumos (tintura, toallas) y, cada tanto, cuenta cuánto le queda. Si algo está por debajo del mínimo, la aplicación lo marca.

Qué hay que programar

Servicio	☐ `GastoServicio.Listar(idNegocio, desde, hasta)`, `Crear`, `Modificar`, `Eliminar`	El tipo de gasto está activo, el importe es mayor a 0 y la fecha no es futura.
Servicio	☐ `InsumoServicio.Listar`, `Crear`, `Modificar`, `CambiarEstado`	Nombre obligatorio y sin repetir; stock y mínimo de 0 o más.
Servicio	☐ `InsumoServicio.DeclararStock(idNegocio, idInsumo, stock)`	La dueña dice cuánto hay; se guarda la fecha. El stock no se descuenta solo (decisión del equipo).
Controller	☐ `GastosController`: `Index(desde, hasta)`, `Crear`, `Editar`, `Eliminar`	Por defecto, el mes actual.
Controller	☐ `InsumosController`: `Index`, `Crear`, `Editar`, `DeclararStock`, `CambiarEstado`	
Vista	☐ `Views/Gastos/Index.cshtml`, `Crear.cshtml`, `Editar.cshtml`; `Views/Insumos/Index.cshtml`, `Crear.cshtml`, `Editar.cshtml`	

Cómo probar que funciona

✓ Declarar un stock menor al mínimo: el insumo aparece marcado en la lista y en el inicio del panel (T28).

T27 Reporte del período

Antes hay que tener hecha: **T25, T26**

PARA QUÉ SIRVE

Entre dos fechas: cuánto se cobró, cuánto se gastó y cuánto quedó, con el detalle por concepto de cobro y por tipo de gasto.

Qué hay que programar

Datos	☐ `PagoDatos.ObtenerTotalesPorConcepto(idNegocio, desde, hasta)`	Nuevo. Suma por concepto, sin anulados.
Datos	☐ `GastoDatos.ObtenerTotalesPorTipo(idNegocio, desde, hasta)`	Nuevo.

Servicio	☐ `ReporteServicio.ObtenerResumen(idNegocio, desde, hasta)`	Junta todo, más la cantidad de atenciones y de ausencias. La fecha "desde" no puede ser mayor que "hasta".
Controller	☐ `ReportesController.Index(desde, hasta)`	Por defecto, el mes actual.
Vista	☐ `Views/Reportes/Index.cshtml`	Tablas simples. Un gráfico es opcional.

Cómo probar que funciona

✓ El total del mes es igual a la suma de la caja de cada día de ese mes.

T28 Pantalla de inicio del panel

Antes hay que tener hecha: **T12, T23, T25, T26**

PARA QUÉ SIRVE

Lo primero que ve la dueña al entrar: los turnos de hoy, lo cobrado hoy, los insumos que faltan y los abonos vencidos.

Qué hay que programar

Servicio	☐ `InicioServicio.ObtenerResumen(idNegocio)`	Reúne datos de los servicios de las tareas anteriores.
Controller	☐ `PanelController.Index()`	A dónde se va después de iniciar sesión.
Vista	☐ `Views/Panel/Index.cshtml`	Cada bloque es un enlace a su pantalla.

Cómo probar que funciona

✓ Cada bloque lleva a la pantalla correcta.

M9 · Avisos

Los avisos no se mandan solos: la aplicación arma el mensaje y la dueña lo envía por WhatsApp con un clic (decisión del equipo).

T29 Mensajes para WhatsApp y agenda de mañana

Antes hay que tener hecha: **T12, T20**

PARA QUÉ SIRVE

Ejemplo de mensaje: "Hola Ana, te recordamos tu turno de Corte mañana a las 15:00 en Peluquería Marina." Al lado, un botón "Copiar" y otro que abre WhatsApp con el mensaje ya escrito.

Qué hay que programar

Servicio	☐ `AvisoServicio.ArmarRecordatorio(turno, cliente, negocio)`	Devuelve el texto del recordatorio.
Servicio	☐ `AvisoServicio.ArmarAvisoSuspension(...)`	Texto para las alumnas de una clase suspendida (T20).
Servicio	☐ `AvisoServicio.ArmarEnlaceWhatsApp(telefono, mensaje)`	Arma `https://wa.me/549...?text=` con el mensaje (usar `Uri.EscapeDataString`).
Servicio	☐ `AgendaServicio.ObtenerManana(idNegocio)`	Turnos de mañana por trabajadora, cada uno con su mensaje.
Controller	☐ `AvisosController.Index(fecha)`	Por defecto, mañana.
Controller	☐ `AgendaController.Manana()`	Agenda de mañana para imprimir, por trabajadora.
Vista	☐ `Views/Avisos/Index.cshtml`, `Views/Agenda/Manana.cshtml`	El botón Copiar usa `navigator.clipboard.writeText` en JavaScript.

Cómo probar que funciona

✓ Tocar el botón de WhatsApp desde el celular: abre el chat con el mensaje escrito.

M10 · Cuenta de cliente

Es opcional: se puede reservar sin cuenta. Una misma cuenta sirve para todos los negocios. Para que nadie use un email ajeno, se confirma con un código que llega por email.

T30 Enviar emails

Antes hay que tener hecha: **Ninguna**

PARA QUÉ SIRVE

Hace falta para mandar los códigos de verificación y de recuperación de contraseña.

Qué hay que programar

Servicio	☐ `EmailServicio.Enviar(para, asunto, cuerpoHtml)`	Con el paquete NuGet MailKit.
Otro	☐ Configuración	Servidor y usuario en `appsettings.json`. La contraseña del email **nunca** va al repositorio: se guarda con `dotnet user-secrets`. Para probar se usa smtp4dev o Mailtrap, que atrapan los emails sin enviarlos de verdad.

Cómo probar que funciona

✓ Mandar un email de prueba y verlo en smtp4dev.

T31 Crear cuenta y verificar el email

Antes hay que tener hecha: **T01, T30**

PARA QUÉ SIRVE

La clienta se registra con email y contraseña. Le llega un código de 6 números que vence en 15 minutos. Cuando lo escribe, la cuenta queda verificada y se conecta sola con las fichas que ya tenía en los negocios con ese email.

Qué hay que programar

Servicio	☐ `CuentaClienteServicio.Registrar(email, contrasena)`	Email libre, contraseña de 8 o más, hash, guarda la cuenta (`InsertarCuentaCliente`), genera el código (`GuardarCodigoVerificacion`) y lo envía.
Servicio	☐ `CuentaClienteServicio.Verificar(idCuentaCliente, codigo)`	Si el código es correcto y no venció, en una `UnidadDeTrabajo` marca el email verificado y conecta las fichas (`VincularCuentaPorEmail`).
Servicio	☐ `CuentaClienteServicio.ReenviarCodigo(idCuentaCliente)`	Genera y manda uno nuevo.
Controller	☐ `ClienteCuentaController.Registrar()`, `Verificar()`, `ReenviarCodigo()`	En la dirección `/MiCuenta/...`. Llevan `[AllowAnonymous]`.

Vista	☐ `Views/ClienteCuenta/Registrar.cshtml`, `Verificar.cshtml`	

Reglas que tiene que revisar el servicio

- El código se genera con `RandomNumberGenerator.GetInt32(0, 1000000)` (no con `Random`, que es predecible).
- Las fichas se conectan recién después de verificar. Si no, alguien podría registrarse con el email de otra persona y ver sus turnos.

Cómo probar que funciona

✓ Un código equivocado o vencido no verifica la cuenta.

T32 Iniciar sesión y recuperar la contraseña (cliente)

Antes hay que tener hecha: **T31**

PARA QUÉ SIRVE
La clienta entra con email y contraseña. Si se la olvidó, pide un código por email y elige una nueva.

Qué hay que programar

Servicio	☐ `CuentaClienteServicio.ValidarCredenciales(email, contrasena)`	Si la cuenta no está verificada, la manda a verificar.
Servicio	☐ `CuentaClienteServicio.SolicitarRecuperacion(email)`	Si el email existe, guarda y manda un código (`GuardarCodigoRecuperacion`).
Servicio	☐ `CuentaClienteServicio.Restablecer(email, codigo, nueva)`	Si el código es correcto y no venció, guarda la nueva contraseña (`ModificarContrasena`).
Controller	☐ `ClienteCuentaController.IniciarSesion()`, `CerrarSesion()`, `OlvideContrasena()`, `Restablecer()`	La cookie lleva el rol `Cliente` y el id de la cuenta.
Vista	☐ `Views/ClienteCuenta/IniciarSesion.cshtml`, `OlvideContrasena.cshtml`, `Restablecer.cshtml`	

Reglas que tiene que revisar el servicio

- Al pedir recuperación, el mensaje es siempre el mismo, exista o no el email.

Cómo probar que funciona

✓ Una clienta conectada no puede entrar a ninguna pantalla del panel de la dueña.

T33 Panel de la clienta

Antes hay que tener hecha: **T15, T32**

PARA QUÉ SIRVE

La clienta ve sus próximos turnos y clases de todos los negocios donde tiene ficha, y puede cancelarlos.

Qué hay que programar

Datos	☐ `ClienteDatos.ObtenerClientesPorCuenta(idCuentaCliente)`	Nuevo. Las fichas conectadas a la cuenta, una por negocio.
Servicio	☐ `MiPanelServicio.ObtenerMisTurnos(idCuentaCliente)`	Por cada ficha, las anotaciones futuras no canceladas, con el nombre del negocio.
Servicio	☐ `MiPanelServicio.Cancelar(idCuentaCliente, idAnotacion)`	Primero verifica que el turno sea de una ficha de esa cuenta; después usa `AnotacionServicio.CambiarEstado`.
Controller	☐ `MiPanelController.Index()`, `Cancelar(idAnotacion)` POST	Con `[Authorize(Roles = "Cliente")]`.
Vista	☐ `Views/MiPanel/Index.cshtml`	

Cómo probar que funciona

✓ Cambiar a mano el número de turno en el formulario no permite cancelar el turno de otra persona.

Parte 4 · Lista para repartir y marcar

Tarea	Qué es	Antes hace falta	Quién la hace	Hecha
T01	Guardar varias cosas juntas (UnidadDeTrabajo) y devolver errores (Resultado)	Ninguna		☐
T02	Inicio de sesión con cookie y controller base del panel	Ninguna		☐
T03	Registrar un negocio nuevo	T01, T02		☐
T04	Iniciar y cerrar sesión	T02, T03		☐
T05	Editar los datos del negocio	T04		☐
T06	Lista de servicios y precios (tarifario)	T04		☐
T07	Precios del abono mensual	T04		☐
T08	Trabajadoras y sus horarios	T04		☐
T09	Tipos de gasto	T03		☐
T10	Fichas de clientes	T04		☐
T11	Calcular los horarios libres	T06, T08		☐
T12	Agenda del día y detalle de un turno	T10		☐
T13	Dar un turno desde el panel	T01, T11, T12		☐
T14	Atender a alguien que llegó sin turno	T13		☐
T15	Confirmar, cancelar y marcar si vino	T12		☐
T16	Reserva desde la página pública	T11, T13		☐
T17	Grilla semanal de clases	T06, T08		☐
T18	Clases de la semana y lista de cada clase	T15, T17		☐
T19	Anotar a alguien en una clase suelta	T18		☐

Tarea	Qué es	Antes hace falta	Quién la hace	Hecha
T20	Suspender una clase	T18		☐
T21	Inscribir a una clienta a un abono	T07, T17		☐
T22	Cobrar el abono	T21		☐
T23	Abonos vencidos, baja y anulación	T22		☐
T24	Cobrar un turno, una clase suelta o una atención sin turno	T13		☐
T25	Caja del día y anular un cobro	T24		☐
T26	Gastos e insumos	T09		☐
T27	Reporte del período	T25, T26		☐
T28	Pantalla de inicio del panel	T12, T23, T25, T26		☐
T29	Mensajes para WhatsApp y agenda de mañana	T12, T20		☐
T30	Enviar emails	Ninguna		☐
T31	Crear cuenta y verificar el email	T01, T30		☐
T32	Iniciar sesión y recuperar la contraseña (cliente)	T31		☐
T33	Panel de la clienta	T15, T32		☐

Para después (fase 2)

- **Enlace "Ver mi turno":** un link privado que llega al reservar, para ver o cancelar el turno sin iniciar sesión. Ya existen `AnotacionDatos.GuardarCodigoEnlace` y `ObtenerAnotacionPorCodigo`.
- **Recordatorios automáticos por email:** necesitan un proceso que corra solo cada hora (`BackgroundService`) y una columna nueva en `Turnos` para no mandar dos veces.

Quedan afuera del proyecto: devoluciones de dinero, reprogramar turnos, puestos y salas, y descuento automático de insumos.

Decisiones para charlar con el grupo

Tema	Pregunta y sugerencia
Anular el cobro de un abono	¿Se corre solo el vencimiento para atrás, o lo corrige la dueña a mano? Sugerencia: a mano, con un aviso en pantalla.
Emails de aviso	¿Se manda un email al reservar o cancelar, además del WhatsApp? Es fácil una vez hecha T30.
Motivo del precio	¿Siempre hay que escribir un motivo si se cobra distinto al precio? La guía dice que sí.
Límite para cancelar	¿Hasta cuántas horas antes puede cancelar la clienta desde su panel? La guía no pone límite.